Structural Reinforcement Learning for Heterogeneous Agent Models

Yucheng Yang, Chiyuan Wang, Andreas Schaab, Benjamin Moll (2025)

Presented by **Karol Jarząbek**

July 2026

Presentation Plan

Huggett Model Setup

Canonical Heterogeneous Agent Environment

- Continuum of households $i \in [0,1]$ subject to idiosyncratic productivity risk $y_{i,t} \sim \mathcal{T}_y(\cdot|y_{i,t-1})$ and aggregate TFP risk $z_t \sim \mathcal{T}_z(\cdot|z_{t-1})$.
- Individual state: $s_{i,t} = (b_{i,t}, y_{i,t})$. Aggregate state: $(\mathbf{g}_t, z_t)$, where $\mathbf{g}_t$ is the cross-sectional distribution (histogram vector).

Individual Optimization Problem

$$\max_{\{c_{i,t}\}} \mathbb{E}_0 \sum_{t=0}^{\infty} \beta^t u(c_{i,t}) \quad \text{s.t.} \quad c_{i,t} + b_{i,t+1} = (1+r_t) b_{i,t} + y_{i,t} z_t, \quad b_{i,t+1} \geq \underline{b}$$

General Equilibrium Market Clearing

$$\int b_t'(b,y,z_t)\,\mathrm{d}G_t(b,y) = 0 \quad \Longrightarrow \quad \text{determines price vector } p_t = \mathcal{P}^*(\mathbf{g}_t, z_t)$$

The Master Equation & Non-Markovian Prices

Rational Expectations Price Functionals

- Equilibrium price functional: $p_t = \mathcal{P}^*(\mathbf{g}_t, z_t)$
- Distribution law of motion (Chapman-Kolmogorov): $\mathbf{g}_{t+1} = \mathbf{A}^{\mathrm{T}}_{\boldsymbol{\pi}_t(z_t, p_t)} \mathbf{g}_t$

The Fundamental Bottleneck: Non-Markovian Prices

- **Micro Preference**: Households care directly about low-dimensional prices p_t, not distribution $\mathbf{g}_t$.
- **Macro Non-Markovianity**: Price p_t is not Markov because $p_{t+1} = \mathcal{P}^*(\mathbf{g}_{t+1}, z_{t+1})$ depends on $\mathbf{g}_{t+1}$. Forecasting p_{t+1} forces agents to forecast $\mathbf{g}_{t+1}$.
- **Master Equation**: Standard Bellman equation forces $\mathbf{g}$ into state space:

$$V(s, \mathbf{g}, z) = \max_c \{u(c) + \beta \mathbb{E}[V(s', \mathbf{g}', z') \mid s, \mathbf{g}, z]\}$$

  Because $\mathbf{g}$ is high-dimensional, global DP suffers from an extreme curse of dimensionality.

Theoretical Foundations: Wold Theorem & RPE

Wold Representation Rationale

By the Wold Representation Theorem, a stationary price process $\{p_t\}$ yields an equivalent VAR(∞) representation:

$$p_{t+1} \sim \mathcal{T}_p(\cdot \mid p_t, p_{t-1}, p_{t-2}, \dots)$$

Observing price history $\{p_\tau\}_{\tau \leq t}$ carries full state information under rational expectations.

Restricted Perceptions Equilibrium (RPE)

- **Restricted State Space**: Household policy $\pi_\theta(s_{i,t}, z_t, p_t, p_{t-1})$ conditions on individual state s, aggregate shock z, current price p_t, and short price history.
- Agents directly learn policy parameters θ that maximize expected lifetime utility over simulated price paths.

SPG Algorithm: Parameterization

Theoretical vs. Sample Objective Function

Theoretical value: $v_{\boldsymbol{\pi}}(s,z,p) = \mathbb{E}\left[\sum_{t=0}^{\infty} \beta^t u(c_{\boldsymbol{\pi}}(s_t,z_t,p_t)) \,\middle|\, s_0 = s, z_0 = z, p_0 = p\right]$. In JAX, we optimize empirical sample objective over N trajectories of length T:

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{N}\sum_{n=1}^{N}\sum_{t=0}^{T} \beta^t (\mathbf{d}_{\boldsymbol{\pi},t}^n)^{\mathrm{T}} \mathbf{u}(\boldsymbol{c}_{\boldsymbol{\theta}}(z_t^n, p_t^n))$$

Representation of $\boldsymbol{\theta}$ as a Vector of Policy Functions

$$\boldsymbol{\theta} = \begin{bmatrix} \pi(z_1, p_1) \\ \vdots \\ \pi(z_K, p_L) \end{bmatrix} = \begin{bmatrix} \pi(s_1, z_1, p_1) \\ \vdots \\ \pi(s_J, z_K, p_L) \end{bmatrix}$$

Here J, K, and L are the numbers of grid points on the s, z, and p grids. That is, the parameter vector $\boldsymbol{\theta}$ is simply the $J \times K \times L$-dimensional vector of values of the policy on the (s, z, p) grid.

SPG Algorithm: contd

Cross-Sectional Path Probability

where $\mathbf{d}^n_{\boldsymbol{\pi},t} = ((\mathbf{A}^n_{\boldsymbol{\pi},0\to t})^{\mathrm{T}})\mathbf{d}_0$ is the cross-sectional distribution of s at time t under policy $\boldsymbol{\pi}$ starting from the initial uniform distribution $\mathbf{d}_0$.

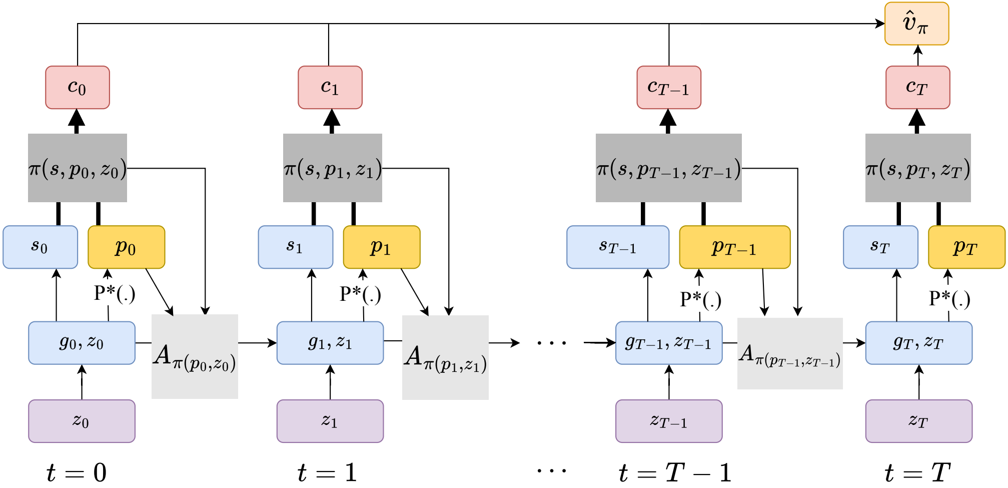
Stochastic Gradient Ascent

We use stochastic gradient ascent (or variants) to update:

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k + \eta_k \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}_k)$$

where η_k is the learning rate at iteration k. A stop-gradient is applied to the price update $p^n_t = \mathcal{P}^*(\mathbf{g}^n_t, z^n_t)$ so that, when computing gradients, prices are treated as exogenous.

SPG Algorithm: contd



The Complete SPG Algorithm

Stochastic Gradient Ascent Formulation

We use stochastic gradient ascent to update:

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k + \eta_k \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}_k)$$

where η_k is the learning rate at iteration k. A stop-gradient is applied to the price update $p_t^n = \mathcal{P}^*(\mathbf{g}_t^n, z_t^n)$ so that, when computing gradients, prices are treated as exogenous.

Steps

1. **Initialize Grid**: Parameterize policy grid $\boldsymbol{\theta}_0$ over (s, z, p).
2. **Simulate Trajectories**: Draw N aggregate shock paths $\{z_t^n\}$. Push forward population distribution $\mathbf{g}_{t+1}^n = \mathbf{A}_{\boldsymbol{\pi}}^{\mathrm{T}} \mathbf{g}_t^n$.
3. **In-Loop Market Clearing**.
4. **AD Gradient Ascent**: Compute exact gradient $\nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}_k)$ via JAX reverse-mode AD and update $\boldsymbol{\theta}_{k+1}$.

Computational Experiments: Runtimes Table

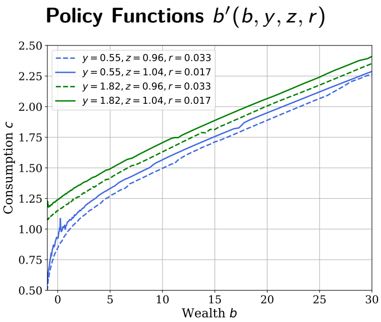
Hardware Setup & Execution Details

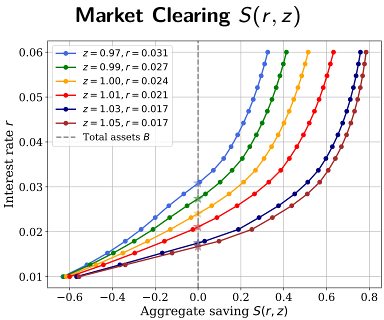
All computational experiments were implemented in JAX and executed on a single NVIDIA A100 GPU.

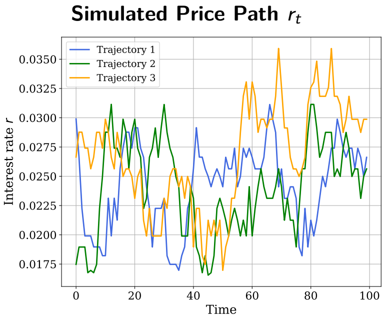
Model	**Average converge epoch**	**# Runs**	**Average Runtime (sec)**
Krusell-Smith	438.4	10	56.55
Huggett with agg. shocks	480.6	10	75.29
HANK with agg. shocks	496.5	10	199.53
Partial equilibrium (Huggett)	289.3	10	39.49

Note: The first column shows the average number of epochs until convergence, and the last column the corresponding average time for a single run.

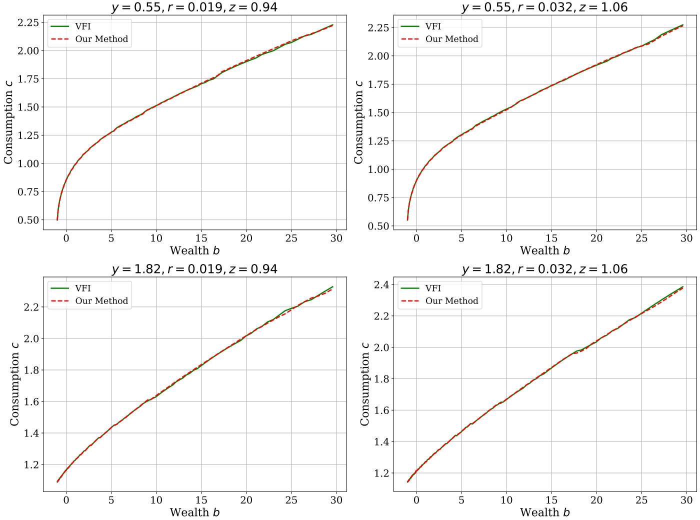
Huggett Model Quantitative Results (Figure 3)



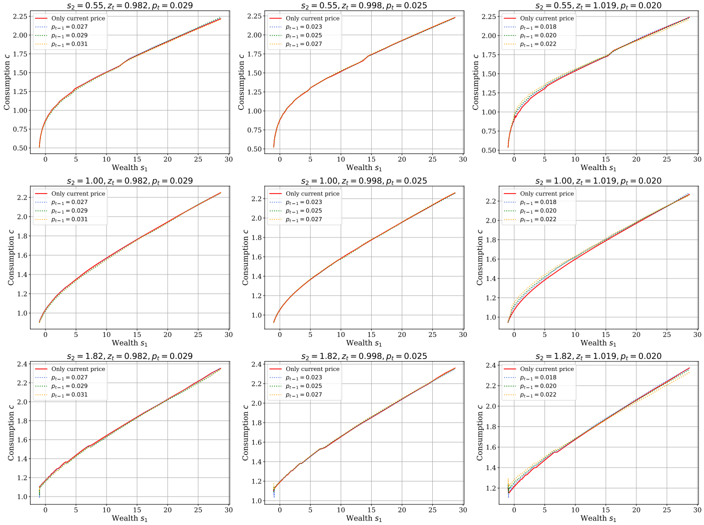




Partial Equilibrium Comparison: SPG vs VFI

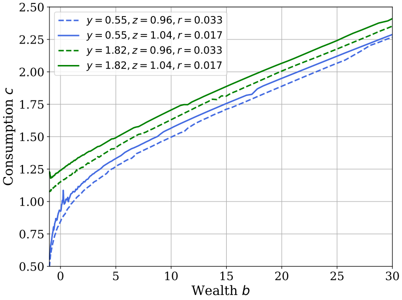


Huggett Model with Price Lags

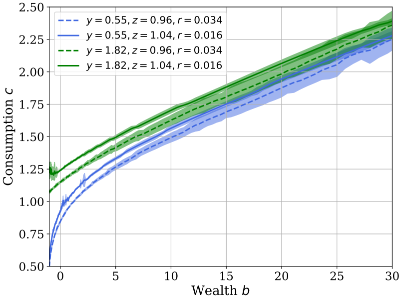


Policy Confidence Intervals: Dependence on Sample Size (N_{sample})

(a) Policy with $N_{\text{sample}} = 512$



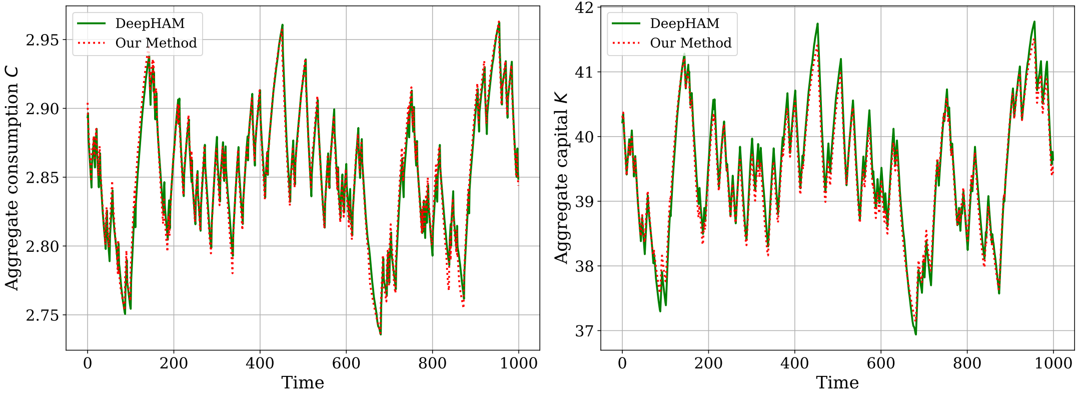
(d) Policy CI with $N_{\text{sample}} = 24$



Interpretation of Wide Confidence Intervals at Small $N_{\text{sample}} = 24$

With a small sample size ($N_{\text{sample}} = 24$), confidence intervals expand significantly at high wealth levels because very few agents visit extreme wealth states in the ergodic distribution, leading to higher Monte Carlo sampling variance in those unvisited regions.

RE Solution Comparison: SPG vs DeepHAM



Equilibrium Trajectory Alignment

Comparing Rational Expectations simulated trajectories generated by DeepHAM (Han et al. 2021) and SPG (Yang et al. 2025) for Krusell-Smith model

Conclusion

1. **Sidestepping the Master equation**: Bypasses the high-dimensional Master Equation by simulating Monte Carlo price paths via Wold's Theorem.
2. **Tabular Grid**: Parameterizes policy grid $\boldsymbol{\theta}$ directly on low-dimensional state space (s, z, p)
3. **Speed**: Solves non-linear HA models with aggregate risk globally in 1–4 minutes on a single NVIDIA A100 GPU
4. `github.com/StructuralRL/SRL`